

LibFlow
Analysis and Design Document : Assignment 1

Student: Costin Alexa
Group: 30431

Table of Contents

Contents

1. Requirements Analysis	3
1.1 Assignment Specification	3
1.2 Functional Requirements	3
1.3 Non-functional Requirements	3
2. Use-Case Model	3
3. System Architectural Design	4

1. Requirements Analysis

1.1 Assignment Specification

The objective of this project is to develop a library management application designed to catalog books and help with user management. The system aims to provide a centralized platform where administrators can manage users (people) and their roles, organize a collection of books with detailed metadata (ISBN, titles), and categorize items through a flexible many-to-many genre classification system.

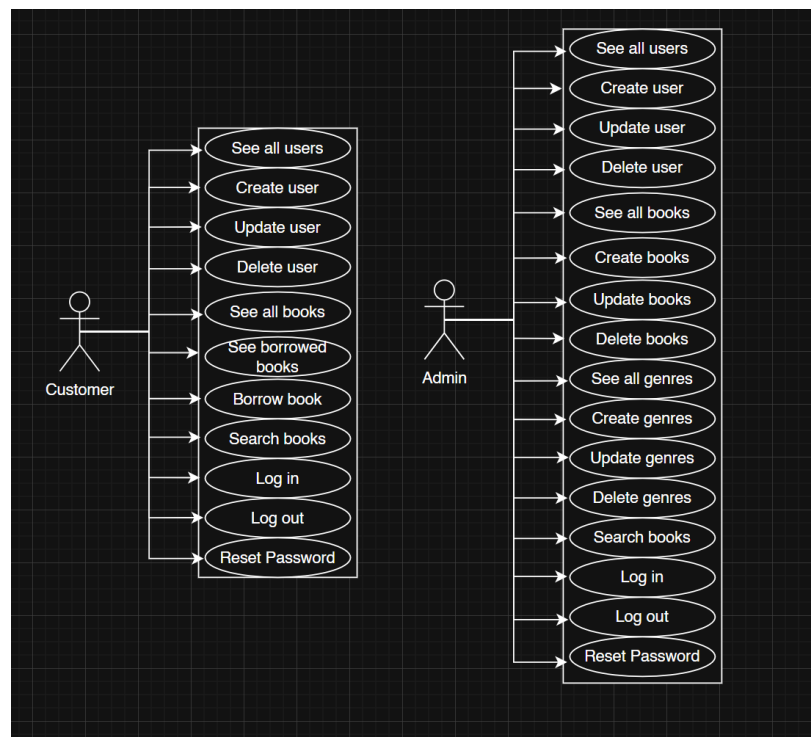
1.2 Functional Requirements

1. Allow the creation, updating, and deletion of user accounts, books and genres.
2. Assign and manage distinct access roles for users (e.g., Customer, Admin).
3. Provide a frontend user interface (UI) for users to interact with the system.

1.3 Non-functional Requirements

1. Implement validation to ensure that all inputs are valid and adhere to the specified format.
2. Use an ORM to handle database interactions.
3. Use a layered architecture for better organization and separation of concerns.
4. Use a database to store all the data.

2. Use-Case Model



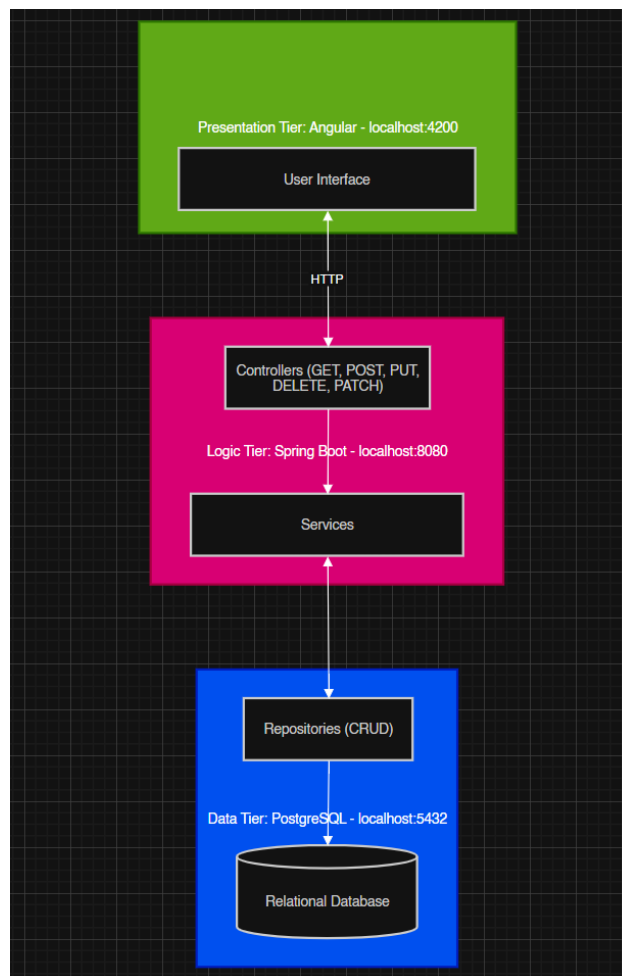
3. System Architectural Design

3.1 Architectural Pattern Description

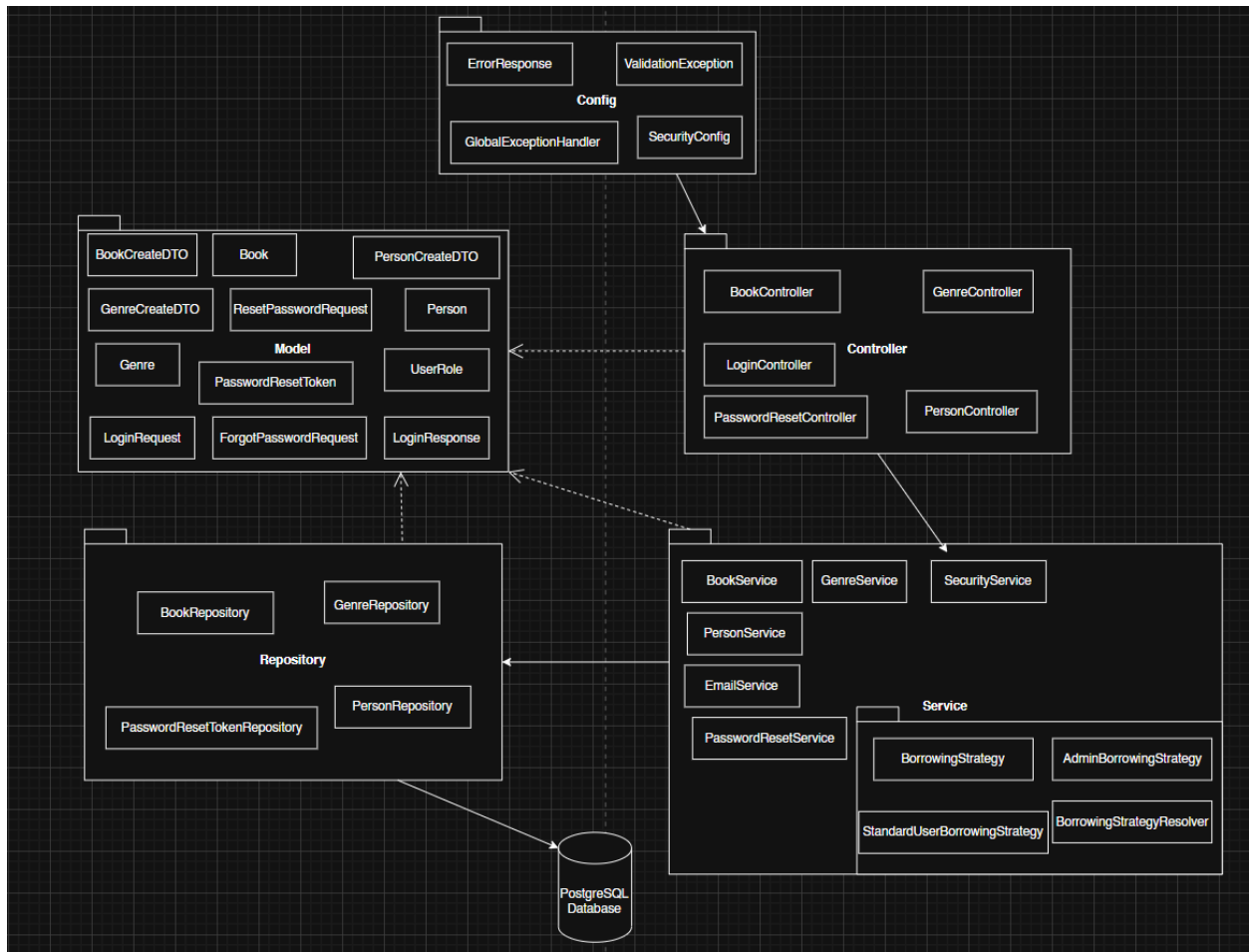
The application is designed in a Layered Architecture divided into 3 tiers:

- **Presentation Tier (Client):** Implemented using Angular 19. This tier handles the user interface and user interactions, communicating with the backend via RESTful APIs.
- **Logic Tier (Server):** Implemented using Java with Spring Boot. This serves as the middle layer that processes business logic, validates data, and coordinates between the UI and the database.
- **Data Tier (Database):** Implemented using PostgreSQL. This tier is responsible for persistent storage of persons, books, and genres.

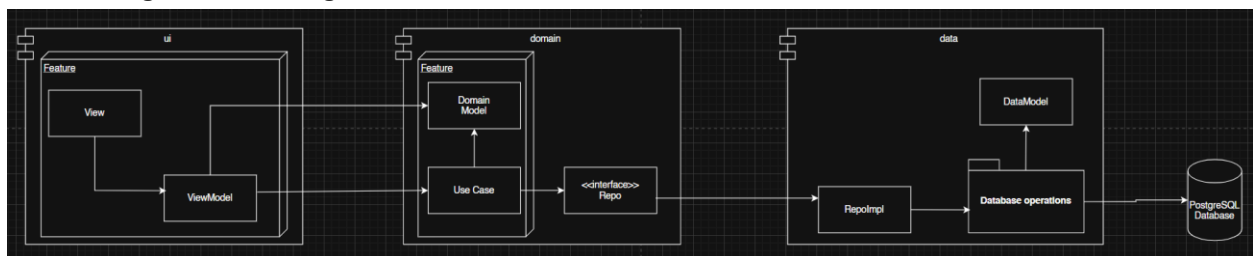
3.2 Diagrams



4. Class Design

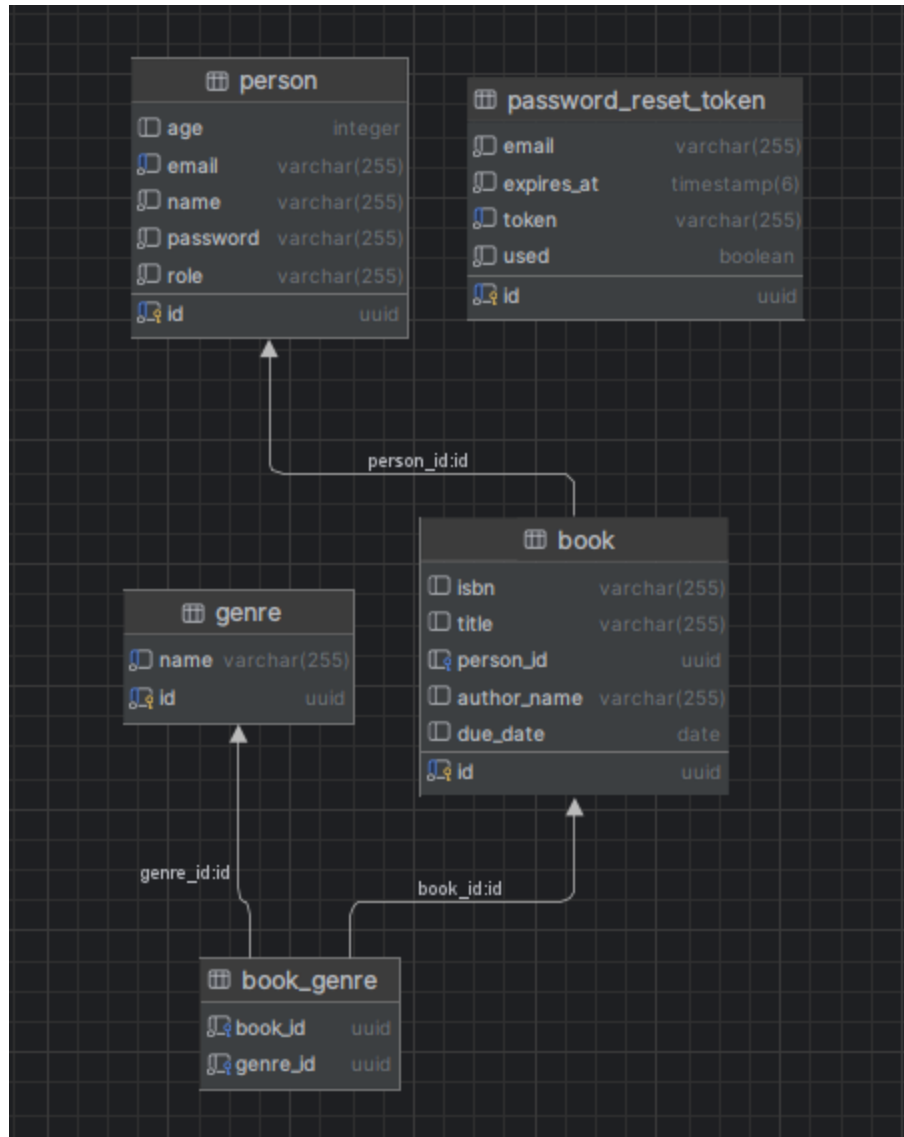


4.1 Package + Class Diagram



5. Data Model

The data model for the library management application consists of five primary entities structured to support user, book and genre management. The relational schema utilizes primary keys (PK) for entity identification and foreign keys (FK) to establish One-to-Many (1:n) and Many-to-Many (n:m) relationships.



6. Changes for the project

Technical Implementation Addendum: Advanced UI/UX Architecture & Non-Trivial Business Logic Engine

1. UI/UX Engineering, Unified Design Tokens, and Accessibility Framework

1.1 Architectural Domain Analysis & Semantic Foundations

Prior to any engineering execution, an interface topology study was conducted against high-scale cataloging and reader-community platforms—specifically **Goodreads**, **Open Library**, **WorldCat**, and **LibraryThing**. The objective was to eliminate arbitrary styling paradigms ("div-soup") in favor of a strictly structured, tokenized presentation layer built upon high-performance semantic HTML layout strategies.

1.2 Centralized CSS Design Tokens (styles.scss)

The presentation architecture completely deprecates inline declarations and hardcoded hex value types. Style properties are derived from primitive-to-semantic design tokens declared natively

within the root DOM boundaries:

- **Functional Color Layering:** Establishes core root variables (--clr-brand-*, --clr-neutral-*) dynamically assigned to functional slots (--bg-page, --text-primary, --border-subtle, --accent).
- **Modular Typography Scale:** Couples an editorial serif display type (*Playfair Display*) with a geometric sans-serif structural body font (*DM Sans*) across an standardized fluid scale spanning --text-xs (11px) through --text-3xl (36px).
- **Layout Padding Ratios:** Employs a deterministic 4px-base modular spacing matrix (--sp-1 to --sp-16) to govern layout padding and flex grid alignments uniformly across views.
- **1.3 Reactive Light/Dark Mode Synchronizer (app.ts & styles.scss)**

Theme shifting is handled client-side without causing content reflows or component reinstantiations:

- **Preference Introspection:** At lifecycle initialization (ngOnInit), the subsystem polls local persistent cache, cascading to a hardware-level media query check (window.matchMedia('(prefers-color-scheme: dark)').matches) if user overrides are absent.
- **DOM Tree Modification:** Utilizing Angular's Renderer2 abstraction layer, the runtime mutates a global HTML node attribute data-theme. This mapping instantly flips functional color assignments inside styles.scss (e.g., swapping background and text steps).
- **1.4 Fluid Responsive Breakdown & Mobile Navigation Shell (app.scss)**

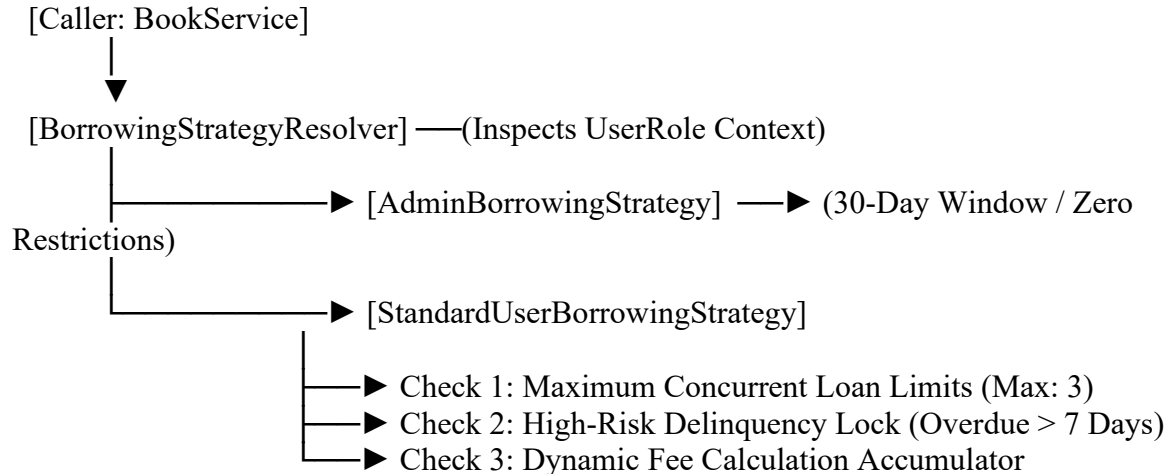
View scaling enforces a strict structural breakpoint schema to support varied computing interfaces:

- **Desktop Configurations:** Interactive pathways and resource navigations map inside a wide layout using fixed grid lines and explicit sidebar distributions.
- **Mobile Adaptations:** For compact viewpoints (\$<640\text{px}\$), the main nav components (.nav-links) compress fully out of layout flows. The screen registers a dedicated click toggle (.hamburger-btn), exposing a sliding overlay layout managed dynamically via touch events.
- **1.5 Web Content Accessibility Guidelines (WCAG) AA Architecture**
- **Focus Indicators:** Explicit browser indicator profiles are suppressed globally to introduce high-contrast focus indicators using customized color parameters (--focus-ring) that preserve focus visibility for keyboard-only actions.
- **Utility Screen-Reader Layer:** Embedded metadata strings are wrapped inside an invisible layout structure (.sr-only), keeping essential system info fully available to screen-readers while maintaining look-and-feel specs visually.
- **ARIA Integrity:** Templates feature rich contextual mappings including live status warnings (aria-live="polite"), container dependencies (aria-labelledby), popup status indications (aria-haspopup="dialog"), and reactive system validations ([attr.aria-invalid]).
- **2. Non-Trivial Business Logic Engine: Polymorphic Rule Evaluation**

Rather than restricting model handling to standard database storage or single CRUD tasks, the core transactional system utilizes an asynchronous **Strategy Design Pattern** pipeline. This engine processes real-time parameter changes, analyzes user profiles, enforces risk rules, and calculates penalties dynamically before committing transactions.

- **2.1 Architectural Components and Strategy Schema**

- BorrowingStrategy (Core behavioral abstraction interface)
- BorrowingStrategyResolver (Contextual injection component that resolves strategy configurations based on metadata)
- StandardUserBorrowingStrategy & AdminBorrowingStrategy (Concrete implementations housing custom behavioral configurations)



- **2.2 Operational Engine Workflows**
- **Step 1: Dynamic Allocation of Target Windows**

When an asset transaction occurs, the core processing engine intercepts the call to resolve the structural behavior rules associated with the user account role configuration. Standard user accounts (CUSTOMER) resolve a **14-day loan window**. Administrative accounts (ADMIN) trigger a 30-day window, proving that system thresholds shift dynamically based on runtime security rules rather than static inputs.

- **Step 2: System Delinquency & Account State Validations**

During a checkout process, the active strategy evaluates historical lending behaviors before executing database changes:

- **Concurrent Limit Verification:** Prevents standard user accounts from exceeding a hard threshold of three concurrent loans.
- **Account Lockout Checks:** The engine evaluates temporal date deltas across existing items. If any asset exceeds its dynamic due date by **more than 7 days**, the account status is updated to locked, throwing a functional system validation exception and blocking the current checkout pipeline.

- **Step 3: Computational Overdue Liabilities Tracking**

When a resource is checked back into the system, the application calculates overdue fine balances before unlinking database data nodes. The system computes outstanding penalties using time delta mathematics (\$0.50 per day past due) based on target return boundaries. This computational data is then logged as user financial history prior to closing the transaction.

- **2.3 Evaluation of Non-Trivial Project Complexity**

The business flow meets advanced criteria through the following architectural practices:

1. **Multi-tiered Logic Chains:** The system operates beyond default form validations, relying on deep calculations across multiple entity relationships and historical dates to process user events.
2. **Decoupled Architectural Patterns:** The Strategy pattern eliminates fragile, hardcoded conditional checks. Expanding system configurations—such as adding a premium or

restricted membership tier—only requires registering a single concrete class, ensuring the core transactional service layer remains clean, maintainable, and isolated from changes.